

1. Goals and Threads

A normal conversation can drift because each turn optimizes locally. A goal introduces a durable finish condition that the agent can revisit across many turns or scheduled continuations. Kundel describes Codex goal mode as persistent work toward a concrete outcome, potentially over long periods.

The key design fact is that the goal prompt is an exit criterion. It should therefore be shorter and more testable than a complete execution script.

2. Good Goal Properties

A useful goal is:

Specific enough to distinguish success from progress.

Verifiable by evidence the agent can actually obtain.

Bounded by invariants the agent must not trade away.

Authorized: it does not imply consequential actions beyond what the user permitted.

Recoverable: after interruption, the current state can be reconstructed from files, notes, or history.

"Make the app faster" is weak. "Reduce p95 API latency for endpoint X below 250 ms in the staging benchmark, with no failing regression tests and no cache TTL increase" is much stronger.

3. Metrics Can Corrupt Goals

A target that can be gamed will eventually invite a cheap path. A bundle-size goal can be met by deleting functionality. A test-coverage goal can be met by trivial assertions. A page-speed goal can be met by deferring work until after the metric window.

Therefore every important metric should be paired with invariants and a human-readable statement of intent.

4. Goal versus Scheduled Work

A goal answers "what state are we trying to reach?" A schedule answers "when should the agent wake up or check again?" Conflating them makes automation harder to reason about. Waiting for an external build, support reply, or benchmark can be scheduled while the same goal remains stable.

The same distinction applies outside coding: the target state and the cadence are different parts of the system.

5. Memories

Codex memories can carry useful context learned from prior work. They reduce repeated explanation, but they are not a replacement for explicit task instructions or current external facts. A remembered preference can guide presentation; a current API version should still be verified.

Treat memory as fallible context with lower authority than a direct present instruction. Do not encode a critical build command only in memory if the repository needs every contributor to know it.

6. Long-Run Recovery

A long-running goal should survive interruption. Keep durable evidence in the repository where appropriate, concise state in notes, and enough pointers to recover older context. If the only record of a crucial decision is a sentence 80 turns ago, the system is fragile even if searchable history can technically find it.

7. Stopping

The hardest part of a goal can be knowing when to stop. Kundel recommends explicit handback criteria. A task that says "keep improving until it's excellent" has no stable terminal state. A better instruction says what checks to perform, how many improvement passes to make, or what threshold ends the loop.

For exploratory work, stopping can be evidence-based rather than numerical: stop after the leading hypotheses are tested and remaining uncertainty is stated. The agent should not invent work merely to stay busy.

8. The Goal Review

Before activating a long-running goal, ask:

What observable state ends it?

What must remain unchanged?

What actions still require approval?

What external event might require waiting?

Where will durable findings be recorded?

What condition means "blocked" rather than "keep trying forever"?

References: Dominik Kundel, "A guide to /goal"; OpenAI, Codex customization and memories; OpenAI, GPT-6 Astra launch.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 17: Goals and Memory

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>